PRESENTASI TUGAS KELOMPOK

Analisis & Implementasi Perbandingan

Layered vs Non-Layered Web Application Architecture

Studi Kasus: Customer Management System (PHP Native, REST API JSON, & MySQL)

1. Pendahuluan & Konsep

- Pengertian Non-Layered & 3-Layer
- secara Netral & Objektif

2. Diagram & Kode Program

- Pemisahan File/Folder, Hubungan
- HTML/CSS/PHP/SQL, & Bukti Kode

3. Demo Web & Alur Proses

- Tampilan CRUD, Validasi Email,
- & Alur Request-Response

4. Tabel Pengujian & Diskusi

- 1 Tabel Utama Hasil Uji Coba,
- Siapa Lebih Unggul, & Alasannya

Pendahuluan: Latar Belakang & Tujuan Perbandingan

Mengenal Dua Pendekatan Arsitektur Aplikasi Web yang Akan Diimplementasikan & Diuji

Dua Pendekatan Arsitektur Aplikasi Web

1. Pendekatan Pertama: Non-Layered Architecture

Aplikasi dikembangkan tanpa pemisahan layer secara tegas. Tampilan (HTML/CSS), proses bisnis (validasi PHP), dan akses database (query SQL) ditempatkan secara terpadu dalam satu file.

2. Pendekatan Kedua: 3-Layer Architecture

Aplikasi dikembangkan dengan memisahkan tanggung jawab secara tegas menjadi tiga lapisan: Presentation Layer, Business Logic Layer, dan Data Layer.

3. Mengapa Perlu Dianalisis & Dibandingkan?

Kedua pendekatan memiliki cara kerja dan karakteristik masing-masing. Oleh karena itu, keduanya diimplementasikan pada studi kasus yang sama untuk diuji secara objektif.

Tujuan & Metodologi Perbandingan

1. Membangun 2 Aplikasi dengan Fitur Identik

Membangun Customer Management System (6 fitur wajib) menggunakan pendekatan Non-Layered (1 file PHP) dan 3-Layer (3 folder + REST API JSON).

2. Membandingkan Struktur File/Folder & Kode

Menunjukkan perbedaan hubungan HTML, CSS, JS, PHP, dan SQL melalui diagram arsitektur serta screenshot kode kedua pendekatan.

3. Menguji & Menilai pada Tabel Perbandingan Akhir

Menguji kedua aplikasi pada 8 unsur (kecepatan buat awal, individu vs tim, penelusuran bug, respons UI, dll.) untuk melihat keunggulan masing-masing secara objektif.

Konsep Pendekatan 1: Non-Layered Architecture

Pengertian, Karakteristik, & Cara Kerja Aplikasi Tanpa 3-Layer (Sesuai Halaman 14–16 PDF)

Pengertian Non-Layered Architecture (Hal. 14 PDF)

1. Definisi Arsitektur Monolitik Sederhana

Disebut juga monolithic application, yaitu pendekatan dengan kode frontend (tampilan), proses bisnis, dan akses database yang tidak dipisahkan secara tegas.

2. Seluruh Proses dalam Satu File PHP (index.php)

Mulai dari: (1) Menampilkan formulir, (2) Menerima input pengguna, (3) Memvalidasi data, (4) Mengakses database MySQL, hingga (5) Menampilkan hasil.

3. Catatan Penting (Halaman 16 PDF)

Aplikasi satu file ini bukan berarti sama sekali tidak memiliki tiga tanggung jawab tersebut. Fungsi tampilan, proses bisnis, dan akses data tetap ada, tetapi menyatu dalam satu file.

6 Tahap Cara Kerja Aplikasi Tanpa 3-Layer (Hal. 15 PDF)

Tahap 1 — Pengguna Mengisi Formulir

Pengguna memasukkan data nama, telepon, email, dan alamat pada form HTML.

Tahap 2 — Browser Mengirim Data ke index.php

Formulir dikirimkan ke file index.php yang sama menggunakan HTTP POST.

Tahap 3 — PHP Memproses Data

Kode PHP di dalam index.php melakukan validasi dan menjalankan query SQL.

Tahap 4 — MySQL Menyimpan Data

Data pelanggan yang valid dimasukkan ke dalam tabel pelanggan di MySQL.

Tahap 5 — PHP Mengambil Data Terbaru

File index.php menjalankan query SELECT untuk mengambil daftar terbaru.

Tahap 6 — Browser Menampilkan Data

Halaman dimuat ulang dan pengguna melihat tabel daftar pelanggan terbaru.

Konsep Pendekatan 2: 3-Layer Architecture

Pengertian Konsep & Penjelasan Detail Ketiga Lapisan (Sesuai Halaman 2–9 PDF)

Pengertian 3-Layer Web Application Architecture (Halaman 2 PDF):

- Konsep pengembangan aplikasi web dengan membagi sistem menjadi TIGA LAPISAN (LAYER) yang memiliki tugas & tanggung jawab berbeda.
- **Tiga lapisan tersebut adalah: (1) Presentation Layer, (2) Application / Business Logic Layer, dan (3) Data Layer.**

1. PRESENTATION LAYER

Lapisan Antarmuka (Frontend)

Pengertian:

Bagian aplikasi yang berinteraksi langsung dengan pengguna.

Tugas Utama:

- Menampilkan informasi (tabel/pesan)
- Menerima masukan form dari user
- Mengirim request ke backend via API

Teknologi yang Digunakan:

- HTML (struktur halaman)
- CSS (tampilan & desain)
- JavaScript (interaksi & Fetch API)
- (Opsional: React, Vue, Angular)

Intinya:

Bertanggung jawab terhadap apa yang dilihat & digunakan oleh pengguna.

2. BUSINESS LOGIC LAYER

Lapisan Logika Bisnis (Backend)

Pengertian:

Bagian yang menangani proses bisnis atau aturan utama aplikasi.

Tugas Utama:

- Menerima permintaan dari frontend
- Memvalidasi kelengkapan & aturan bisnis (misal: cek email unik)
- Mengambil keputusan & kirim JSON

Teknologi yang Digunakan:

- PHP Native (atau Laravel)
- Python, Java, Node.js, C#
- Komunikasi via REST API & JSON

Intinya:

Bertanggung jawab terhadap bagaimana aplikasi bekerja & mengambil keputusan.

3. DATA LAYER

Lapisan Pengelolaan Data (Database)

Pengertian:

Bagian yang berisi database dan mekanisme akses data.

Tugas Utama:

- Menyimpan data baru (INSERT)
- Mengambil data (SELECT / Cari)
- Memperbarui data (UPDATE)
- Menghapus data (DELETE)

Teknologi yang Digunakan:

- MySQL / MariaDB (PDO PHP)
- PostgreSQL, SQL Server, Oracle
- MongoDB

Intinya:

Bertanggung jawab terhadap penyimpanan dan pengelolaan data aplikasi.

Tanggung Jawab Layer & Perbandingan Karakteristik Dasar

Pembagian Tugas 3-Layer (Hal. 11 PDF) & Perbedaan Konsep Tanpa 3-Layer vs 3-Layer (Hal. 16 PDF)

Tabel Tanggung Jawab Ketiga Layer (Hal. 11 PDF)

Aspek	Presentation	Business Logic	Data Layer
Fungsi Utama	Tampilan (UI)	Proses bisnis	Penyimpanan data
Interaksi dgn	Pengguna	Frontend & DB	Backend
Contoh Tekno	HTML, CSS, JS	PHP, Python	MySQL
Folder Proyek	presentation/	business-logic/	data-access/
Tanggung Jawab	Menampilkan data	Mengolah data	Menyimpan data

5 Prinsip Dasar Arsitektur 3-Layer (Hal. 12 PDF):

1. Separation of concerns
2. Maintainability
3. Scalability
4. Security (Isolasi akses DB)
5. Reusability (REST API JSON)

Komunikasi: Presentation <-> REST API JSON <-> Logic <-> Data

Perbandingan Karakteristik Dasar (Hal. 16 PDF)

Aspek	Tanpa 3-Layer	Dengan 3-Layer
Struktur	Sederhana (menyatu)	Terpisah per layer
File program	Bisa satu file (index.php)	Beberapa komponen/folder
Frontend	Menyatu dengan PHP	Terpisah (presentation/)
Business logic	Menyatu dgn kode aplikasi	Dipisahkan (business-logic/)
Database access	Langsung dari file PHP	Melalui komponen data access
Pemeliharaan	Lebih sulit jika kompleks	Lebih terstruktur
Pengembangan	Cepat untuk aplikasi kecil	Lebih terorganisasi
Pengujian	Diuji terpadu di 1 file	Dapat dipisahkan per layer

Studi Kasus: Customer Management System

6 Fitur Wajib Aplikasi & Desain Tabel Database MySQL (Halaman 18–19 PDF)

6 Fitur Wajib yang Diimplementasikan

1. Tambah Pelanggan

Memasukkan data pelanggan baru (nama, email, telepon, alamat).

2. Tampil Pelanggan

Menampilkan seluruh daftar data pelanggan perusahaan.

3. Edit Pelanggan

Mengubah informasi pelanggan yang sudah terdaftar.

4. Hapus Pelanggan

Menghapus data pelanggan dari database MySQL.

5. Pencarian Pelanggan

Mencari pelanggan berdasarkan Nama atau Email.

6. Validasi Data

Memeriksa kelengkapan kolom dan keunikan alamat email.

File Database SQL: customer_management.sql

```
CREATE DATABASE IF NOT EXISTS customer_management;
USE customer_management;

CREATE TABLE pelanggan (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nama VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    telepon VARCHAR(25) NOT NULL,
    alamat TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Sudah Terisi 5 Data Awal Pelanggan:
INSERT INTO pelanggan VALUES
('Andi Pratama', 'andi@nusantara.co.id', ...),
('Budi Santoso', 'budi@teknologimaju.id', ...),
('Citra Lestari', 'citra@kreasidigital.com', ...),
('Dewi Anggraini', 'dewi@sinergiglobal.co.id', ...),
('Eko Prasetyo', 'eko@mitraniaga.id', ...);
```

Aplikasi 1: Non-Layered Architecture (Diagram & Struktur File)

Ketentuan Halaman 20 PDF: HTML, CSS, Validasi PHP, & Query SQL Menyatu dalam 1 File index.php

DIAGRAM ARSITEKTUR 1: NON-LAYERED (STRUKTUR FILE & HUBUNGAN KOMPONEN)

Tidak Ada Pemisahan File/Folder Layer: HTML, CSS, Logika PHP, dan Query SQL Menyatu di 1 File (index.php)

STRUKTUR FOLDER & FILE (NON-LAYERED)

```
[Folder] 1-non-layered-app/
 └── [File] index.php   (HANYA 1 FILE!)
      ├── [Baris 16-39]  Koneksi PDO MySQL
      ├── [Baris 44-160] Validasi PHP & Query SQL
      ├── [Baris 168-200] Kode Desain <style> CSS
      └── [Baris 202-304] Kode Tampilan HTML
```

Karakteristik Arsitektur Non-Layered:

1. Satu File Terpadu (index.php)

HTML, CSS, PHP, dan SQL dikelola langsung dalam 1 file utama.

2. Tampilan & Logika Menyatu

Formulir HTML dan tabel berada di file yang sama dengan kode PHP.

3. Validasi & Query Langsung

Pemeriksaan input dan query SQL langsung dijalankan saat submit.

4. Komunikasi Form HTTP Biasa

Menggunakan pengiriman form standar (POST/GET) tanpa REST API.

ISI DI DALAM FILE: 1-non-layered-app/index.php (MENYATU)

BAGIAN TAMPILAN: HTML + CSS (Menyatu di Bagian Bawah index.php)

- Tag <style> CSS mengatur warna & tabel langsung di dalam index.php
- Tag <form> HTML & tabel <table> bercampur dengan <?php foreach (...) ?>

BAGIAN PROSES BISNIS: Validasi PHP (Menyatu di Bagian Tengah index.php)

- Blok if ($_SERVER['REQUEST_METHOD'] === 'POST') menerima input form
- Memeriksa kelengkapan kolom kosong & format email langsung di file yang sama

BAGIAN AKSES DATA: Koneksi PDO & Query SQL (Menyatu di index.php)

- `new PDO('mysql:host=localhost;dbname=customer_management', 'root', '')`
- Query SELECT, INSERT, UPDATE, DELETE ditulis langsung bersama kode form

Query SQL Langsung

Data Tabel pelanggan

DATABASE MYSQL: customer_management (File: database/customer_management.sql)

- Tabel `pelanggan`: id (PK), nama, email (UNIQUE), telepon, alamat, created_at
- **Diakses secara langsung dari file utama index.php menggunakan koneksi PDO MySQL**

Screenshot Code Aplikasi 1 (Non-Layered Architecture)

Bukti Konsep Non-Layered pada 1-non-layered-app/index.php (Menyatu dalam 1 File)

SCREENSHOT CODE 1: KONSEP NON-LAYERED ARCHITECTURE (1 FILE MONOLITIK)

File: 1-non-layered-app/index.php — Koneksi Database, Validasi Proses Bisnis, Query SQL, CSS, dan HTML Menyatu dalam 1 File

1-non-layered-app/index.php (Seluruh Kode Menyatu dalam 1 File)

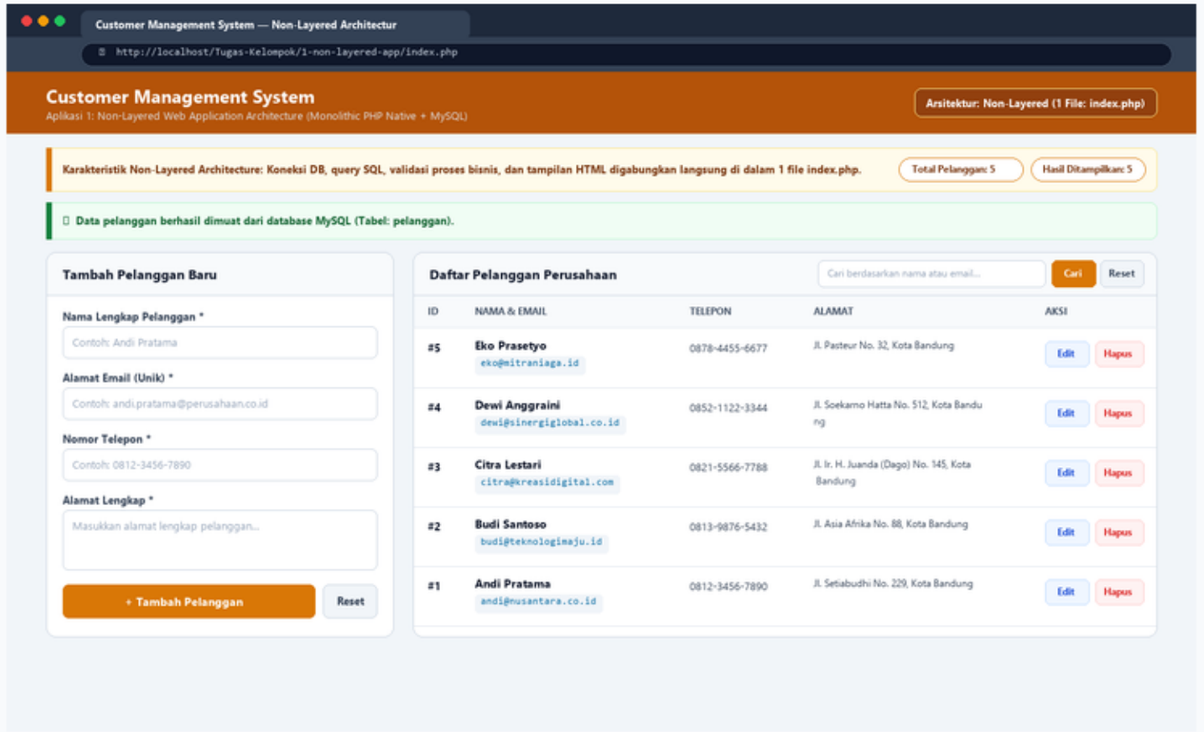
NON-LAYERED (MONOLITIK)

```
// ========================================================================
// BAGIAN 1: KONEKSI LANGSUNG KE DATABASE MYSQL (Di dalam index.php)
// ========================================================================
$koneksi = new PDO("mysql:host=localhost;dbname=customer_management", "root", "");

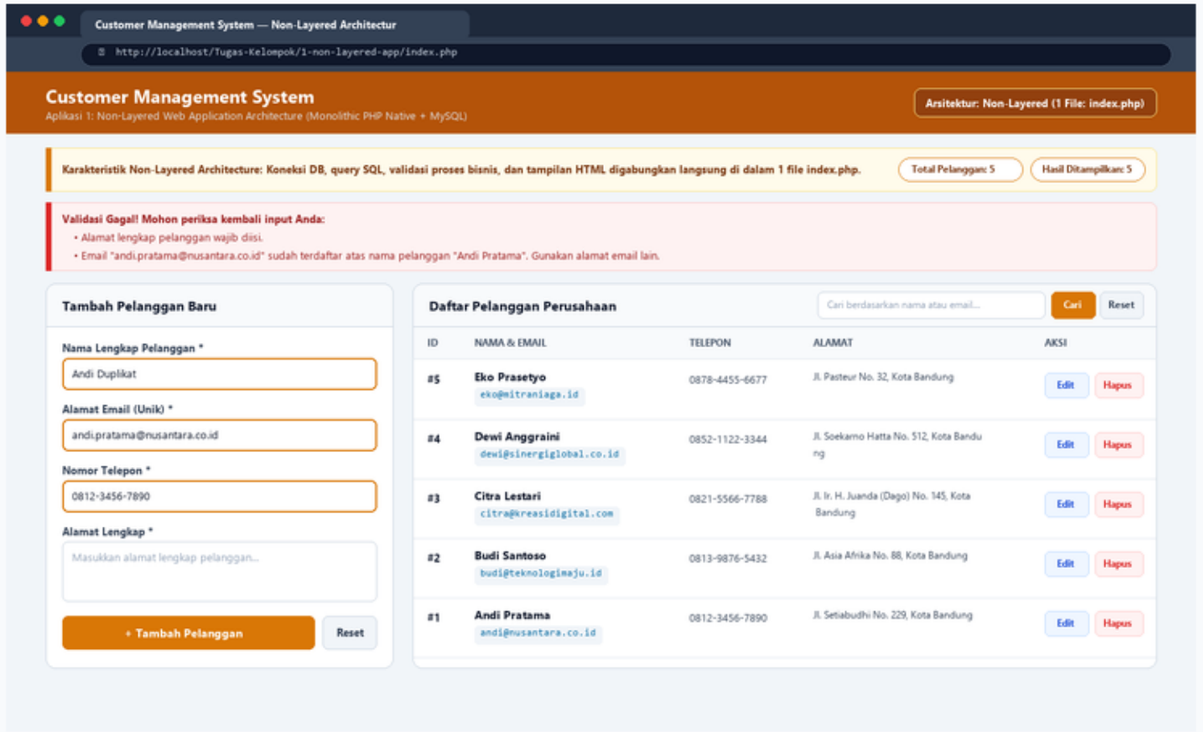
// ========================================================================
// BAGIAN 2: PROSES BISNIS (VALIDASI) & QUERY CRUD (Menyatu di index.php)
// ========================================================================
if (isset($_GET['hapus'])) {
    $stmt = $koneksi->prepare("DELETE FROM pelanggan WHERE id = ?");
    $stmt->execute([(int) $_GET['hapus']]);
}
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // A. Validasi Kelengkapan Data
    if ($nama === '' || $email === '' || $telepon === '' || $alamat === '') {
        $pesan_error[] = "Semua kolom pelanggan wajib diisi.";
    }
    // B. Validasi Keunikan Email (Langsung query SQL di file tampilan)
    $stmtCek = $koneksi->prepare("SELECT id FROM pelanggan WHERE LOWER(email) = ?");
    $stmtCek->execute([$email]);
    if ($stmtCek->fetch()) {
        $pesan_error[] = "Email sudah terdaftar di database!";
    }
    // C. Eksekusi Query INSERT / UPDATE Langsung
    $stmt = $koneksi->prepare("INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES (?,?,?,?)");
    $stmt->execute([$nama, $email, $telepon, $alamat]);
}
// Query Tampil & Pencarian Pelanggan
$daftar_pelanggan = $koneksi->query("SELECT * FROM pelanggan ORDER BY id DESC")->fetchAll();
?>
<!-- ========================================================================
     BAGIAN 3: TAMPILAN HTML & CSS (Menyatu langsung di bawah kode PHP)
     ======================================================================== -->
<!DOCTYPE html><html lang="id"><head><style> /* Kode CSS di sini */ </style></head>
```

Screenshot Tampilan Web Aplikasi 1 (Non-Layered)

Hasil Demo Fitur CRUD, Pencarian, & Validasi Email Unik pada 1-non-layered-app/index.php



Tampilan Utama & Tambah Pelanggan (index.php)



Tampilan Validasi Kolom Kosong & Email Unik

Aplikasi 2: 3-Layer Architecture (Pemisahan Folder & File)

Ketentuan Halaman 21 PDF: Pemisahan Tegas presentation/ <-> business-logic/ <-> data-access/ via REST API JSON

DIAGRAM ARSITEKTUR 2: 3-LAYER ARCHITECTURE (PEMISAHAN FOLDER & FILE)

Pemisahan Tegas ke 3 Folder: presentation/ (HTML/CSS/JS) <-> business-logic/ (PHP API/Service) <-> data-access/ (PDO SQL)

LAYER 1: PRESENTATION LAYER

Folder: 2-layered-app/presentation/

File: index.html (Struktur Halaman)
- Menampilkan form input & tabel pelanggan
- **Murni HTML (0% kode PHP / 0% query SQL)**

File: assets/css/style.css (Desain UI)
- File CSS terpisah untuk mengatur tampilan
- Di-link oleh index.html (<link rel='stylesheet'>)

File: assets/js/app.js (Interaksi & Fetch)
- Mengirim & mengambil data ke Backend
- Menggunakan fetch() REST API & JSON
- Memperbarui isi tabel tanpa reload halaman

Tugas Folder presentation/:
Hanya mengurus tampilan & interaksi user.

HTTP Request

Format JSON

JSON Response

LAYER 2: BUSINESS LOGIC LAYER

Folder: 2-layered-app/business-logic/

File: api/customers.php (REST API Endpoint)
- Pintu komunikasi antara JS dan PHP
- Menerima input JSON (json_decode)
- Memanggil CustomerService.php
- Mengirim hasil JSON (json_encode)

File: CustomerService.php (Aturan Bisnis)
- **Fungsi validasiData():**
  1. Cek kelengkapan nama, email, telp, alamat
  2. Cek format email (FILTER_VALIDATE_EMAIL)
  3. Cek keunikan email ke CustomerRepository
- **Bebas dari kode HTML & bebas dari SQL!**

Tugas Folder business-logic/:
Mengatur logika bisnis, validasi, & REST API.

Panggil Fungsi

Return Array

LAYER 3: DATA LAYER

Folder: 2-layered-app/data-access/

File: CustomerRepository.php (SQL)
- `findAll()` -> `SELECT * FROM pelanggan`
- `search()` -> `SELECT ... WHERE LIKE`
- `findByEmail()` -> Cek email unik di DB
- `create()` -> `INSERT INTO pelanggan`
- `update()` -> `UPDATE pelanggan SET`
- `delete()` -> `DELETE FROM pelanggan`

File: Database.php (Koneksi MySQL)
- Membuka koneksi PDO ke MySQL
- Database: customer_management
- Digunakan oleh CustomerRepository

Tugas Folder data-access/:
Satu-satunya folder yang akses MySQL.

DATABASE MYSQL: customer_management

`File: database/customer_management.sql`

- Tabel `pelanggan` (5 data awal)
- **Aman & terisolasi di belakang Data Layer**

RINGKASAN PEMISAHAN FILE/FOLDER PADA 3-LAYER ARCHITECTURE:

```
[Folder 1] 2-layered-app/presentation/     --> index.html + style.css + app.js (Frontend/UI)
[Folder 2] 2-layered-app/business-logic/   --> CustomerService.php + api/customers.php (Backend/Logic)
[Folder 3] 2-layered-app/data-access/      --> Database.php + CustomerRepository.php (Database/SQL)
```

Screenshot Code Aplikasi 2 (3-Layer Architecture)

Bukti Pemisahan Kode ke 3 Folder: app.js (Presentation), CustomerService.php (Logic), & CustomerRepository.php (Data)

SCREENSHOT CODE 2: KONSEP 3-LAYER ARCHITECTURE (TERPISAH 3 FOLDER LAYER)

Pemisahan Tegas: Presentation Layer (app.js) <-> Business Logic Layer (CustomerService.php & api) <-> Data Layer (CustomerRepository.php)

presentation/app.js — LAYER 1: PRESENTATION

```
1   // Folder: 2-layered-app/presentation/
2   // Murni Frontend (Tanpa PHP & Tanpa SQL)
8   const URL_API = '../business-logic/api/customers.php';
9
11  // 1. Ambil Data via GET REST API (JSON)
12  async function muatDataPelanggan(kataKunci = '') {
18    const response = await fetch(url);
19    const hasil = await response.json();
30    tbody.innerHTML = hasil.data.map(...);
45  }
46
48  // 2. Kirim Data via POST/PUT JSON
60  const response = await fetch(url, {
61    method: idEdit ? 'PUT' : 'POST',
62    headers: {'Content-Type':'application/json'},
63    body: JSON.stringify(dataKirim)
64  });
66  const hasil = await response.json();
```

CustomerService.php — LAYER 2: BUSINESS LOGIC

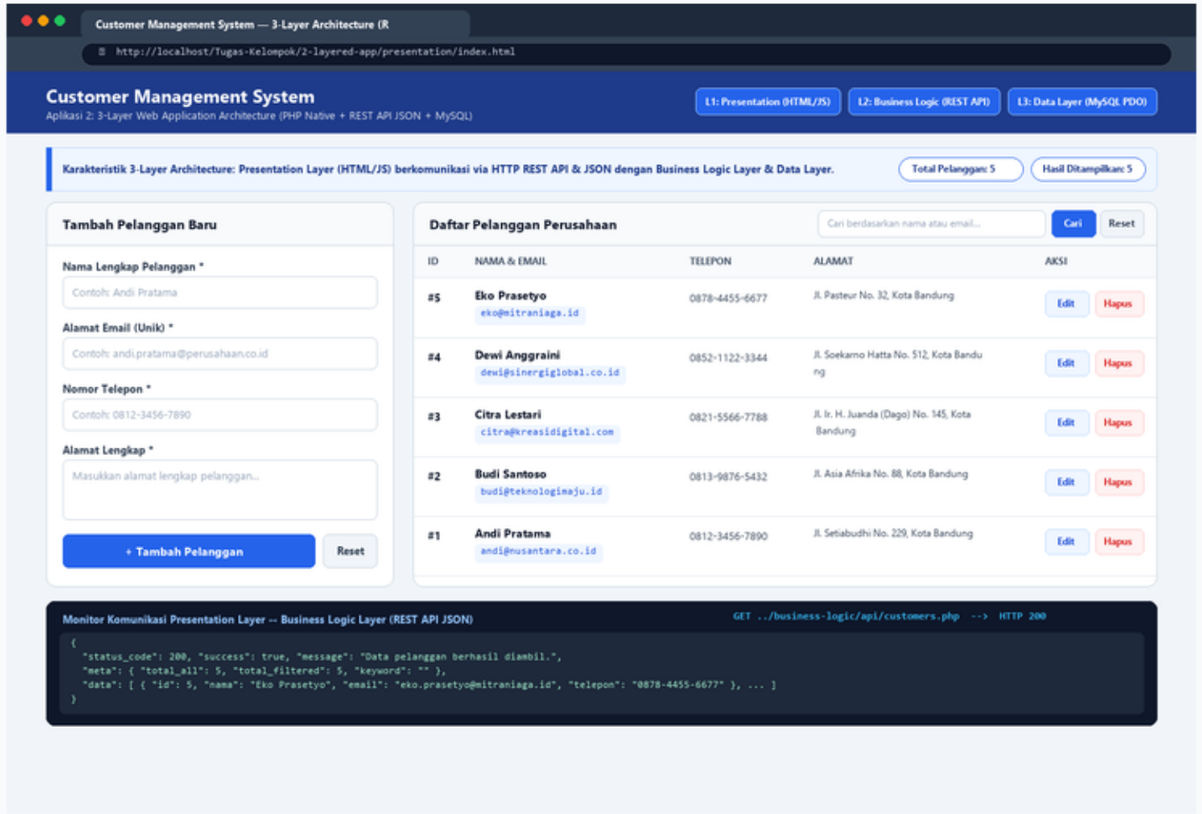
```
1   // Folder: 2-layered-app/business-logic/
2   // Aturan Bisnis, Validasi, & REST API JSON
14  class CustomerService {
56    public function createCustomer($input) {
58      $data = $this->rapikanInput($input);
59      $errors = $this->validasiData($data);
62      if (!empty($errors)) {
63        return ['status_code' => 422,
64                'success' => false,
65                'errors' => $errors];
67      }
70      $baru = $this->repo->create($data);
72      return ['status_code' => 201,
73              'success' => true,
74              'data' => $baru];
76    }
150   // Validasi kelengkapan & email unik
151   private function validasiData($data) {...}
180 }
```

CustomerRepository.php — LAYER 3: DATA LAYER

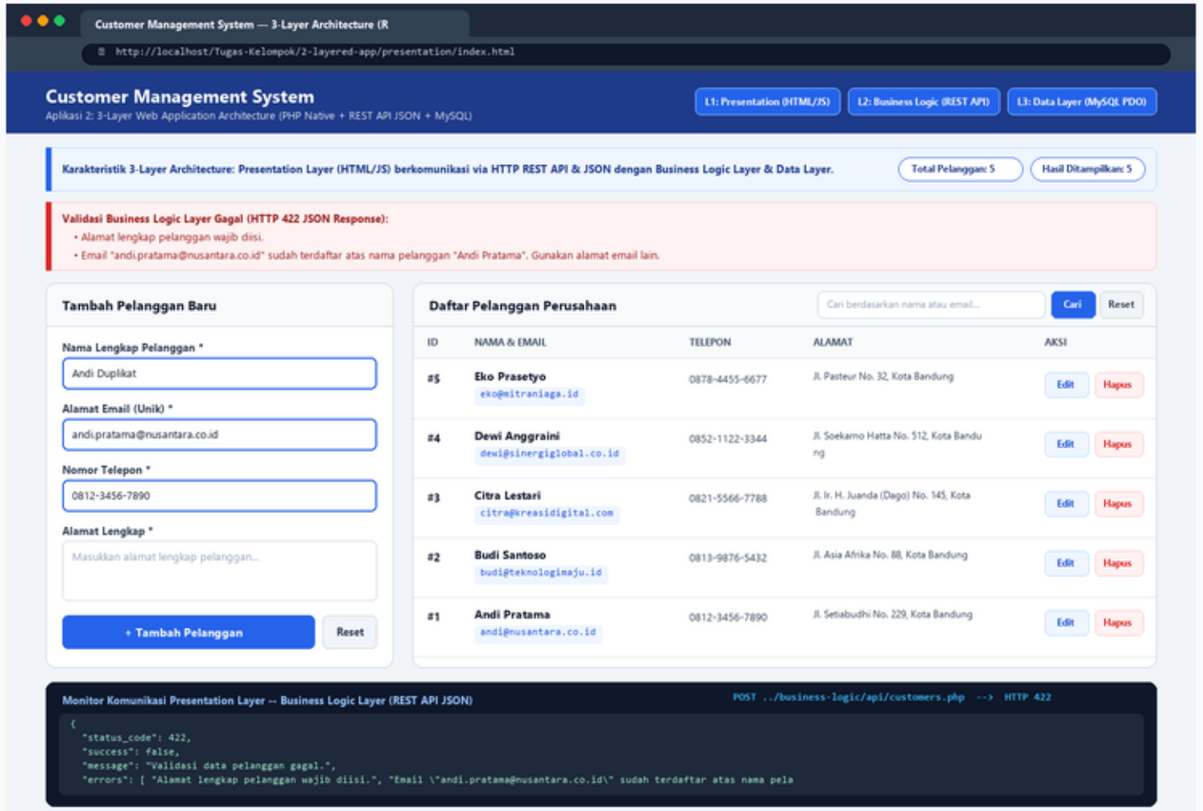
```
1   // Folder: 2-layered-app/data-access/
2   // Khusus Koneksi PDO & Query SQL MySQL
12  class CustomerRepository {
22    public function findAll() {
24      return $this->db->query(
25        'SELECT * FROM pelanggan ORDER BY id DESC'
26      )->fetchAll();
27    }
46    public function findByEmail($email) {
52      $stmt = $this->db->prepare(
53        'SELECT * FROM pelanggan WHERE email = ?'
54      );
57    }
60    public function create($data) {
62      $stmt = $this->db->prepare(
63        'INSERT INTO pelanggan VALUES (?,?,?,?)'
64      );
65      $stmt->execute([...]);
70    }
95  }
```

Screenshot Tampilan Web Aplikasi 2 (3-Layer REST API)

Tampilan Antarmuka Terpisah beserta Bukti Pertukaran Data REST API & JSON



Dashboard 3-Layer + Monitor JSON (HTTP 200 OK)



Respons Validasi Business Logic Layer (HTTP 422 JSON)

Perbandingan Alur Proses Request & Response

Ringkasan Cara Kerja di Balik Layar Setelah Demo Kedua Aplikasi (6 Tahap vs 7 Tahap REST API)

PERBANDINGAN ALUR EKSEKUSI: NON-LAYERED vs 3-LAYER ARCHITECTURE

Studi Kasus: Proses Tambah Pelanggan & Validasi Keunikan Email pada Customer Management System

PENDEKATAN 1: NON-LAYERED ARCHITECTURE (MONOLITIK)

Tahap 1 — Input Form HTML
Pengguna mengisi form di index.php lalu menekan tombol Submit.

↓

Tahap 2 — HTTP POST Form-Urlencoded
Browser mengirim form ke index.php (memicu pemuatan ulang halaman).

↓

Tahap 3 — Validasi Prosedural di index.php
Blok if ($_POST) memeriksa field kosong & menjalankan query cek email.

↓

Tahap 4 — Eksekusi INSERT Langsung
File index.php langsung mengeksekusi query INSERT INTO pelanggan.

↓

Tahap 5 — Query Ulang & Render HTML Utuh
File index.php menjalankan SELECT * lalu mengembalikan seluruh HTML.

PENDEKATAN 2: 3-LAYER ARCHITECTURE (REST API & JSON)

Tahap 1 — Presentation Layer (app.js)
Menangkap submit form & mengirim HTTP POST JSON via fetch() async.

↓

Tahap 2 — REST API Controller (api/customers.php)
Menerima JSON payload & meneruskannya ke CustomerService.

↓

Tahap 3 — Business Logic Layer (CustomerService.php)
Memvalidasi kelengkapan data, format email, & memanggil cek duplikasi.

↓

Tahap 4 — Data Layer (CustomerRepository.php)
Menjalankan Prepared Statement findByEmail() & create() ke MySQL.

↓

Tahap 5 — JSON Response (HTTP 201 / 422)
Backend mengirim JSON; Presentation Layer memperbarui tabel DOM.

Tabel Hasil Pengujian & Perbandingan Akhir

Perbandingan Unsur Uji Coba, Tanpa Layer vs 3-Layer, Siapa yang Lebih Unggul, beserta Alasannya

Unsur yang Diuji Coba	Tanpa Layer (Non-Layered)	Menggunakan 3-Layer	Lebih Unggul	Alasan Utama
1. Pemisahan File/Folder	Menyatu di 1 file index.php	Terpisah tegas di 3 folder layer	**3-Layer**	**Kode rapi & tanggung jawab tiap file jelas**
2. Kecepatan Buat Awal	Sangat cepat (cukup 1 file)	Lebih lama (buat API & 3 layer)	**Non-Layered**	**Praktis untuk aplikasi kecil/sederhana**
3. Individu vs Kerja Tim	Hanya cocok dikerjakan 1 orang	Bisa dibagi paralel ke 3 orang/tim	**3-Layer**	**Tiap anggota kerjakan folder berbeda tanpa bentrok**
4. Penelusuran Error (Bug)	Sulit dicari karena kode campur	Mudah dilacak sesuai foldernya	**3-Layer**	**Langsung tahu letak bug (UI / Logic / SQL)**
5. Kecepatan Respons UI	Lambat (Full Page Reload)	Cepat (Fetch JSON tanpa reload)	**3-Layer**	**Hanya bertukar data JSON yang ringan**
6. Keamanan Edit Desain	Rawan merusak kode PHP/SQL	Aman (HTML/CSS terpisah penuh)	**3-Layer**	**Ubah tampilan tidak menyentuh kode database**
7. Penambahan Fitur Baru	File index.php makin panjang	Tinggal tambah fungsi per layer	**3-Layer**	**Mudah dikembangkan tanpa merusak kode lama**
8. Reusabilitas (Mobile)	Tidak bisa (output berupa HTML)	Bisa dipakai Web & Android/iOS	**3-Layer**	**REST API menghasilkan format standar JSON**

Jawaban 5 Pertanyaan Diskusi (Halaman 17 PDF)

Analisis Perbandingan Berdasarkan Pengalaman Membangun Kedua Aplikasi

1. Mana yang lebih mudah dibuat?

Non-Layered lebih cepat dibuat di awal untuk aplikasi kecil/individu karena seluruh kode cukup ditulis dalam 1 file index.php.

2. Mana yang lebih mudah diperbaiki saat terjadi kesalahan (bug)?

3-Layer jauh lebih mudah ditelusuri karena bug langsung terisolasi sesuai foldernya (apakah di presentation/, business-logic/, atau data-access/).

3. Mana yang lebih mudah dikembangkan saat fitur bertambah?

3-Layer jauh lebih unggul karena mendukung kerja tim secara paralel dan endpoint REST API JSON-nya bisa dipakai ulang oleh aplikasi lain.

4. Apa risiko jika seluruh kode ditempatkan dalam satu file?

Kode tercampur aduk (spaghetti code), edit desain HTML rawan merusak query SQL, rawan bentrok saat kerja tim, dan sulit melacak bug.

5. Apakah aplikasi tanpa 3-layer selalu lebih buruk?

Tidak selalu. Untuk tugas individu atau web kecil sederhana, Non-Layered lebih cepat. Namun untuk sistem bisnis & kerja tim, 3-Layer mutlak lebih unggul.

Kesimpulan Akhir & Rekomendasi

Kapan Memilih Non-Layered Architecture vs Kapan Memilih 3-Layer Architecture?

Kapan Non-Layered Lebih Unggul?

- Unggul pada KECEPATAN PEMBUATAN AWAL.
- Cocok untuk aplikasi yang sangat sederhana (1 halaman).
- Cocok jika dikerjakan oleh 1 ORANG (Individu).
- Cocok untuk prototipe cepat / tugas latihan dasar.

Kelemahan Utama:

- Sulit dikerjakan tim (bentrok di 1 file index.php).
- Sulit menelusuri bug ketika baris kode makin panjang.
- Edit desain HTML berisiko merusak query SQL.

Mengapa 3-Layer Unggul di 7 dari 8 Unsur?

- Pemisahan file/folder sangat rapi (presentation, logic, data).
- Sangat ideal untuk KERJA TIM secara paralel.
- Penelusuran kesalahan (debugging) sangat cepat & terarah.
- Respons UI lebih cepat (Fetch JSON tanpa reload halaman).
- Aman saat mengubah desain HTML/CSS tanpa menyentuh SQL.
- Mudah dikembangkan ketika fitur terus bertambah.
- Endpoint REST API JSON dapat dipakai ulang oleh Mobile.

TERIMA KASIH — SESI TANYA JAWAB & DISKUSI

Seluruh Source Code, Database SQL, Diagram, & Dokumentasi telah tersedia lengkap.